

Building a dashboard like the one in your images requires combining **HTML/CSS** for the look, **PHP** for the logic, and **XAMPP** to run the server.

Since you want to prevent copying and screenshots of PDFs, please note: **It is impossible to 100% block screenshots** (users can always take a photo with their phone), but we can make it very difficult using specific tools.

Step 1: Set up your XAMPP Environment

1. Open the **XAMPP Control Panel** and click **Start** for both Apache and MySQL.
 2. Navigate to C:\xampp\htdocs.
 3. Create a new folder named e-archive.
 4. Inside e-archive, create these folders:
 - uploads/ (to store the PDFs)
 - css/ (for your styles)
 - js/ (for your scripts)
-

Step 2: The Database (MySQL)

To track "Number of Views" and "Manage Users," you need a database.

1. Go to <http://localhost/phpmyadmin>.
 2. Create a database named cpsu_archive.
 3. Create a table for research_papers with columns: id, title, author, department, file_path, and views (set default to 0).
-

Step 3: Create the Dashboard Structure (PHP + HTML)

Create a file named dashboard.php. Use the sidebar and card layout from your screenshots.

PHP

```
<!DOCTYPE html>
<html>
<head>
  <title>e-Archive Admin</title>
  <link rel="stylesheet" href="css/style.css">
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body>
  <div class="sidebar">
    
    <a href="dashboard.php" class="active">Dashboard</a>
    <a href="upload.php">Upload Research</a>
    <a href="users.php">Manage Users</a>
  </div>

  <div class="main-content">
    <header>
      <input type="text" placeholder="Search...">
    </header>

    <h1>Administrator Console</h1>
```

```

<div class="stats-cards">
  <div class="card">Total Repositories: <?php echo $total_count; ?></div>
  <div class="card">Monthly Views: <?php echo $view_count; ?></div>
</div>

<canvas id="viewsChart"></canvas>
</div>

<script>
  // Simple Chart.js logic for your "Thesis Views per Month"
  const ctx = document.getElementById('viewsChart');
  new Chart(ctx, {
    type: 'line',
    data: {
      labels: ['Jan', 'Feb', 'Mar', 'Apr'],
      datasets: [{ label: 'Agriculture', data: [12, 19, 3, 5] }]
    }
  });
</script>
</body>
</html>

```

Step 4: PDF Upload and Security

To handle the "Upload New Research" screen from your image:

1. **PHP Upload Script:** Use `move_uploaded_file()` in PHP to save the PDF to your uploads/ folder.
 2. **Preventing Copy/Paste:** In your CSS, add this to the PDF viewer container:
 CSS


```
.pdf-container {
  user-select: none; /* Prevents text highlighting */
  -webkit-user-select: none;
}
```
 3. **Preventing "Print/Save As":** Instead of using a standard link, use a library like **PDF.js**. It renders the PDF into a "Canvas" (an image), which makes it impossible for users to highlight or right-click text.
-

Step 5: Tracking Views

To count views accurately, every time a user clicks "View" on a paper, run a simple PHP query:

```

PHP
// update_views.php
$id = $_GET['id'];
$sql = "UPDATE research_papers SET views = views + 1 WHERE id = $id";
mysqli_query($conn, $sql);

```

Summary of Tools to Use:

- **PHP:** To connect to the database and handle file uploads.
- **MySQL:** To store the user list and view counts.
- **Chart.js:** The best "simple" way to create the graphs shown in your screenshot.
- **PDF.js:** To display your files safely without allowing text copying.

Would you like me to provide the specific PHP code for the "Upload Paper" form logic?